



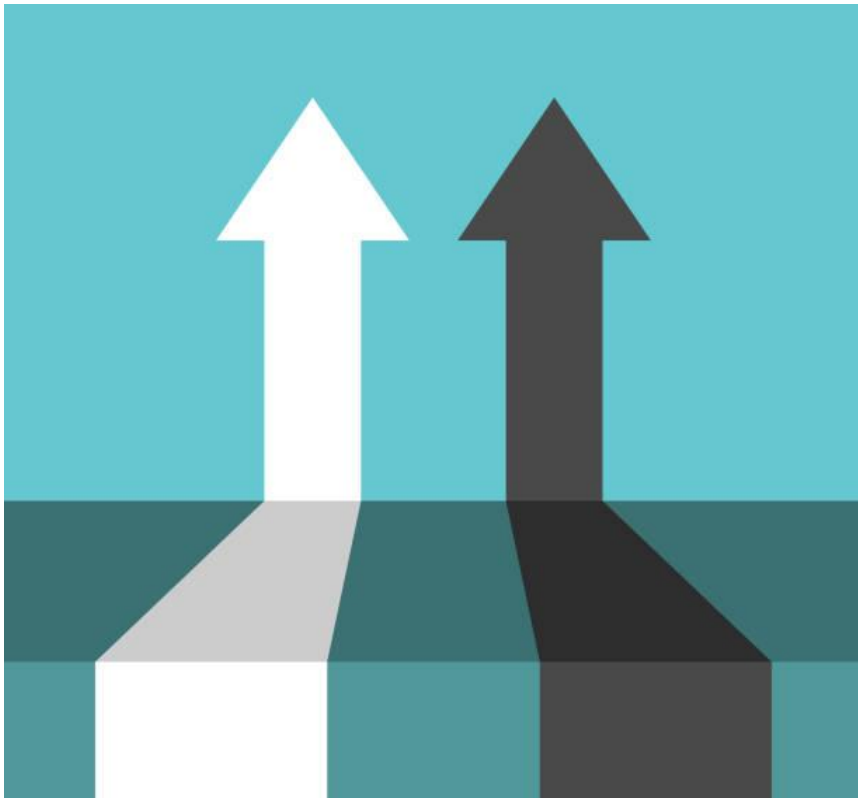
Cómputo Paralelo

y en la Nube

Profesor:

Dr. J. Octavio Gutiérrez García

octavio.gutierrez@itam.mx



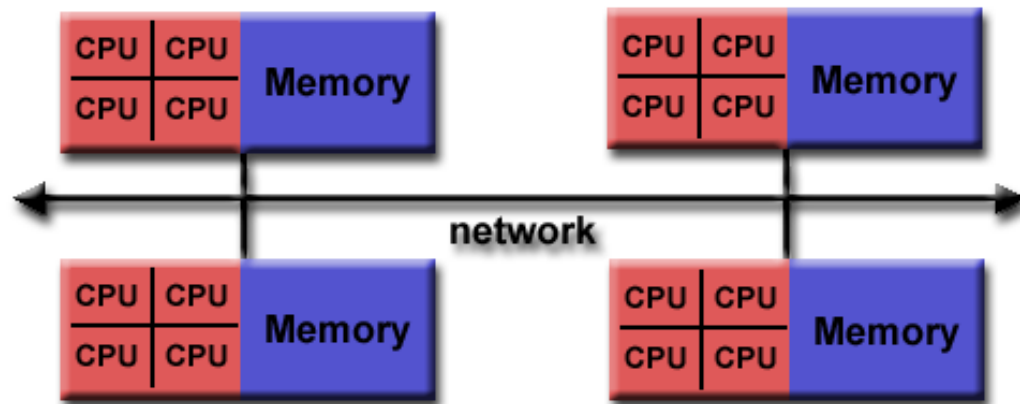
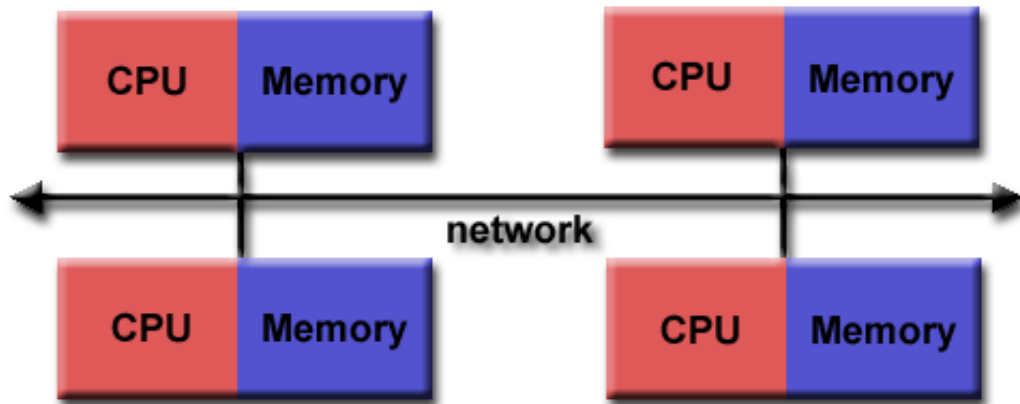


Modelo Paso de Mensajes

MPI

(Message Passing Interface)

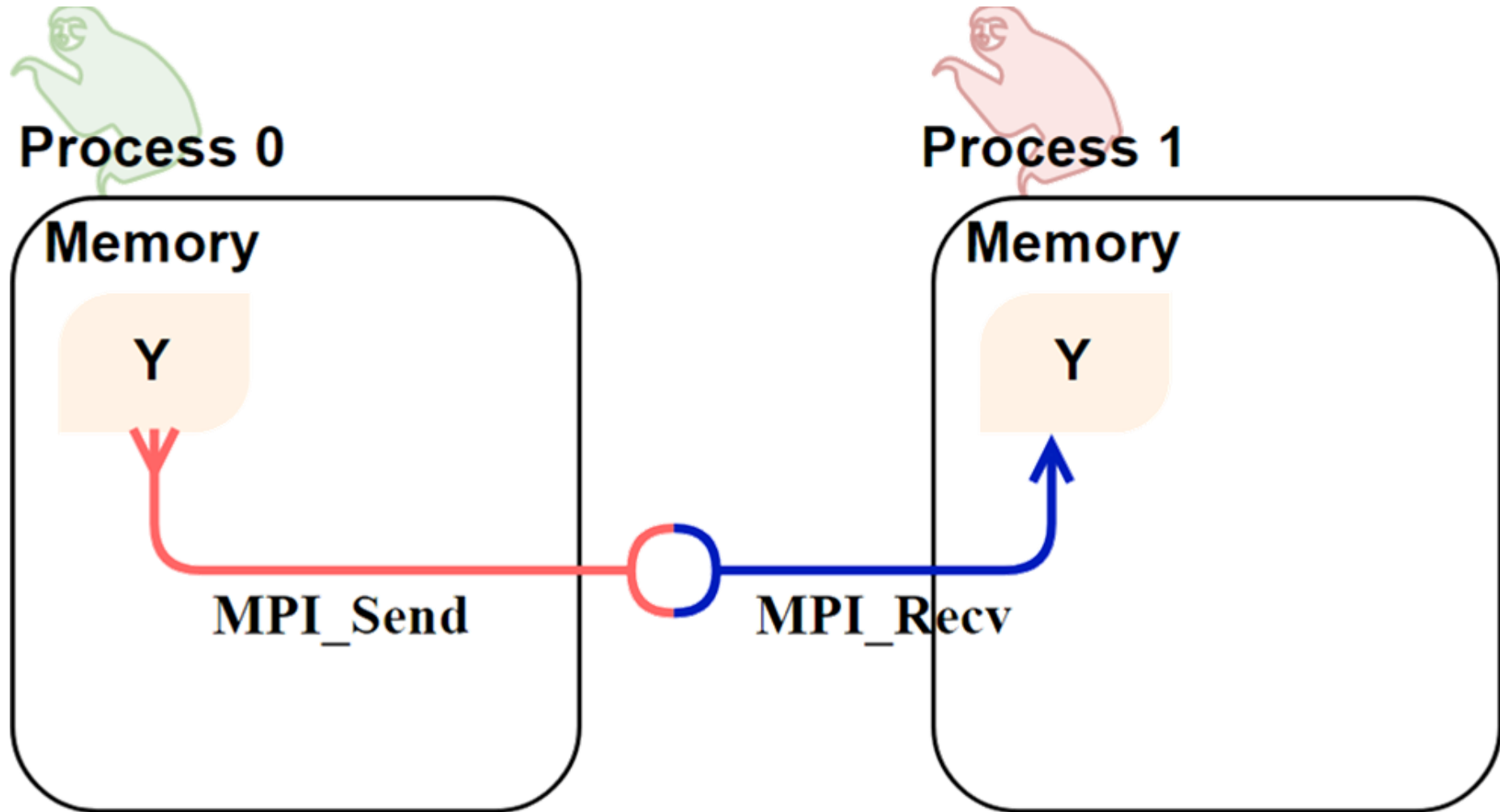
MPI diseñado *originalmente* para Memoria Distribuida



Al día de hoy MPI corre (casi) en cualquier tipo de arquitectura

- Memoria distribuida
- Memoria compartida
- Híbrido

MPI – paso de mensajes



Por qué usar **MPI**

- Rendimiento
- Estandarización/Portabilidad
- Funcionalidad



Instalación de MPI

Windows

- <https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi>
<https://www.youtube.com/watch?v=-gxwT-eAfvU>

MAC/OS

- *Usando Homebrew* shell\$ brew install openmpi
ó
- *Usando MacPorts:* shell\$ port install openmpi

Linux

<https://www.comrevo.com/2019/10/How-to-install-MPI-on-Linux-Ubuntu-and-Fedora.html>
<https://docs.open-mpi.org/en/main/installing-open-mpi/quickstart.html>

Instalación de MPI

Windows

<https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi>

msmpisetup.exe

C:\msmpi\

msmpisdk.msi

C:\msmpi\Sdk

set msmpi

MSMPI_BENCHMARKS=C:\msmpi\Benchmarks\

MSMPI_BIN=C:\msmpi\Bin\

MSMPI_INC=C:\msmpi\Sdk\Include\

MSMPI_LIB32=C:\msmpi\Sdk\Lib\x86\

MSMPI_LIB64=C:\msmpi\Sdk\Lib\x64\

Configuración de *vscode* para MPI

- Crea un Proyecto en VS Code llamado **initial_mpi**
- Descarga **mpi_hello_world.cpp** e incluyelo en tu Proyecto
- Actualiza el **includePath**

```
#include errors detected. Please update  
your includePath. Squiggles are disabled  
for this translation unit  
(C:\Users\Octavio\Desktop\initial_mpi\hello  
(1696)
```

```
cannot open source file "mpi.h" C/C++(1696)
```

[View Problem](#) [Quick Fix... \(Ctrl+.\)](#)

```
#include "mpi.h"
```

Configuración de **vscode** para MPI

- Actualiza el **includePath**

Include path

An include path is a folder that contains header files (such as `#include "myHeaderFile.h"`) that are included in a source file. Specify a list of paths for the IntelliSense engine to use while searching for included header files. Searching on these paths is not recursive. Specify `**` to indicate recursive search. For example, `${workspaceFolder}/**` will search through all subdirectories while `${workspaceFolder}` will not. If on Windows with Visual Studio installed, or if a compiler is specified in the `compilerPath` setting, it is not necessary to list the system include paths in this list.

One include path per line.

```
${workspaceFolder}/**  
${MSMPI_INC}
```

Equivale a poner C:\msmpi\Sdk\Include\

Configuración de *vscode* para MPI (Windows)

Terminal > Configure Default Build Task.

De las opciones elige:

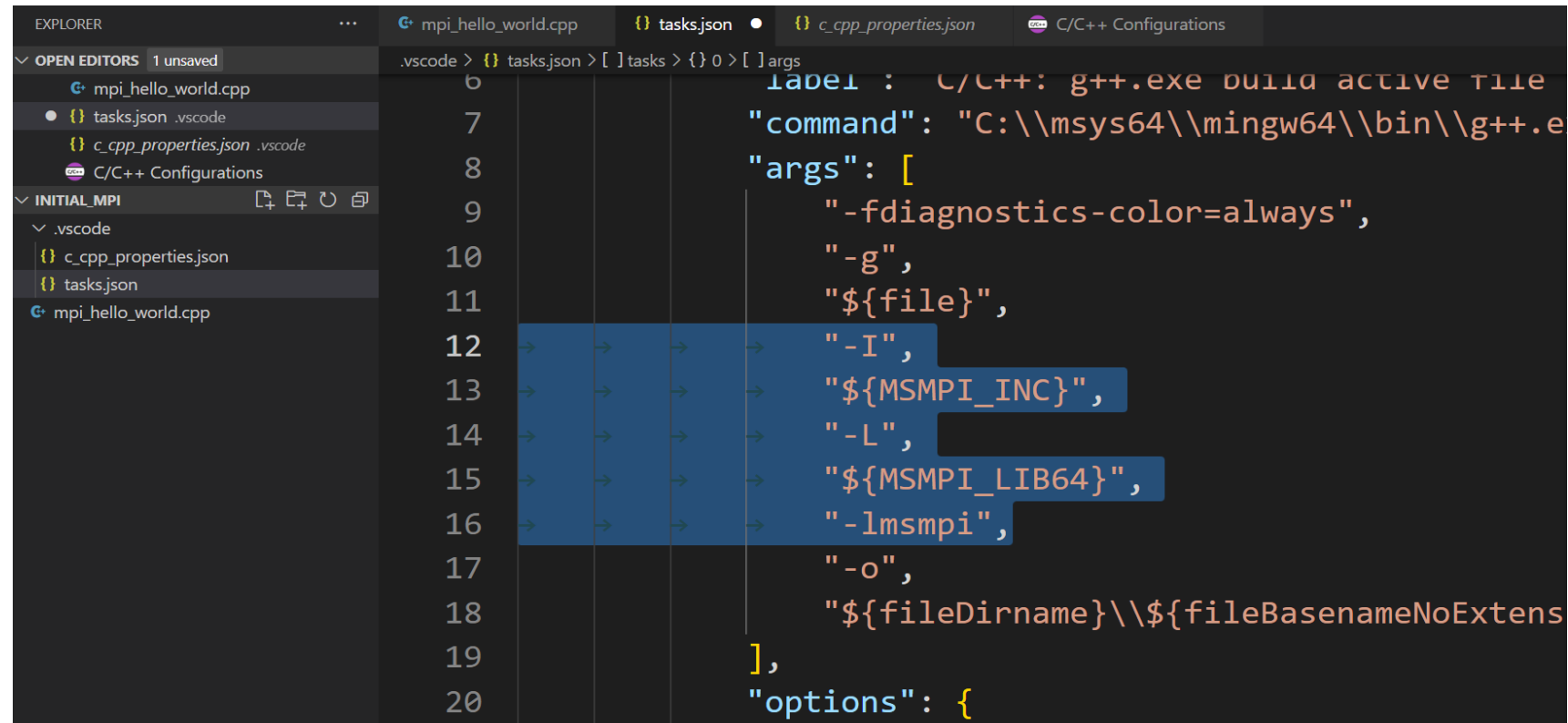
C/C++: g++ build active file.

Lo anterior crea *tasks.json*

Configuración de **vscode** para MPI (Windows)

Dentro de tasks.json en sección de args, justo debajo de “\${file}” incluye los siguientes 5 argumentos

"-I",
"\${MSMPI_INC}",
"-L",
"\${MSMPI_LIB64}",
"-lmsmpi",



The screenshot shows the VS Code interface with the Explorer sidebar on the left and the Editor view on the right. The Explorer sidebar shows the file structure with 'tasks.json' selected under the '.vscode' folder. The Editor view displays the content of 'tasks.json', which is a JSON file defining build tasks. The 'args' array is highlighted with a blue selection box, showing the following arguments: `"-I",`, `"${MSMPI_INC}",`, `"-L",`, `"${MSMPI_LIB64}",`, and `"-lmsmpi",`. The 'command' is set to `"C:\\msys64\\mingw64\\bin\\g++.exe"`. The 'options' object is partially visible at the bottom.

```
.vscode > {} tasks.json > [ ] tasks > {} 0 > [ ] args
6
7
8
9
10
11
12 → → → → → "label": "C/C++: g++.exe build active file"
13 → → → → → "command": "C:\\msys64\\mingw64\\bin\\g++.exe"
14 → → → → → "args": [
15 → → → → →   "-fdiagnostics-color=always",
16 → → → → →   "-g",
17 → → → → →   "${file}",
18 → → → → →   "-I",
19 → → → → →   "${MSMPI_INC}",
20 → → → → →   "-L",
    "${MSMPI_LIB64}",
    "-lmsmpi",
    "-o",
    "${fileDirname}\\${fileBasenameNoExtension}.exe",
  ],
  "options": {
```

Mac OS: mpicc / mpirun

Configuración de **vscode** para **MPI** (Windows)

Terminal > Run Build Task

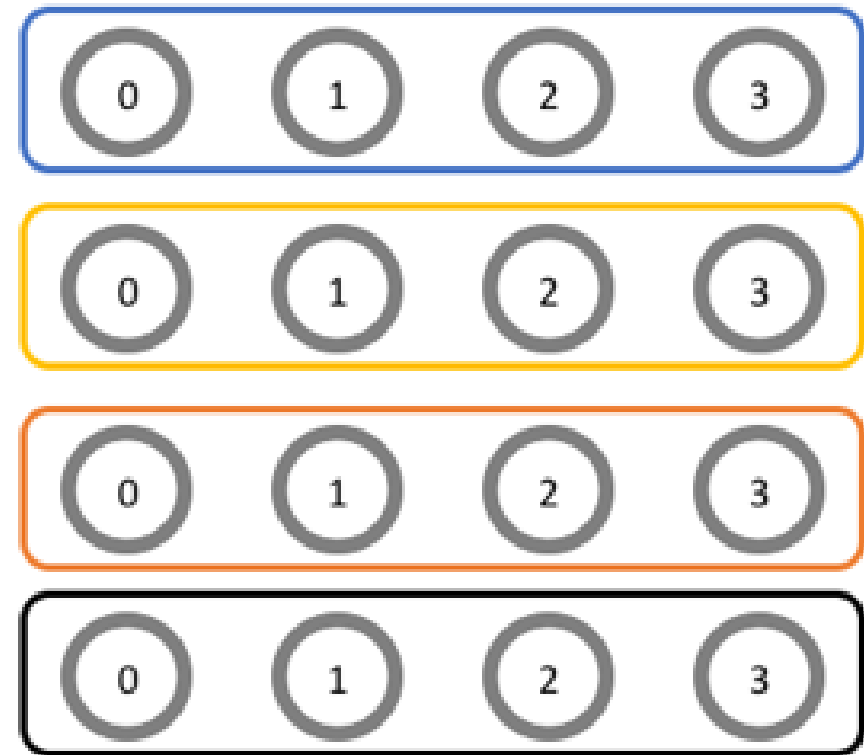
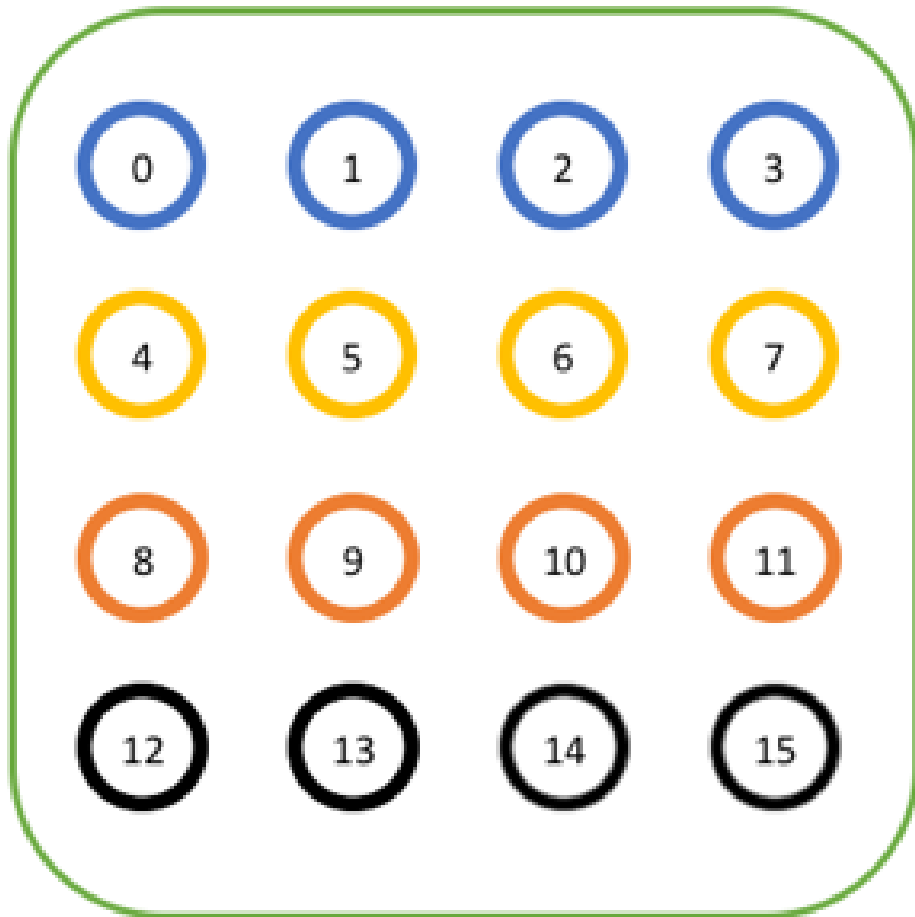
Lo anterior crea el ejecutable **mpi_hello_world.exe**, el cual está listo para ejecutarse vía consola usando usando **mpiexec**:

```
mpiexec mpi_hello_world.exe
```

```
mpiexec -n 4 mpi_hello_world.exe
```

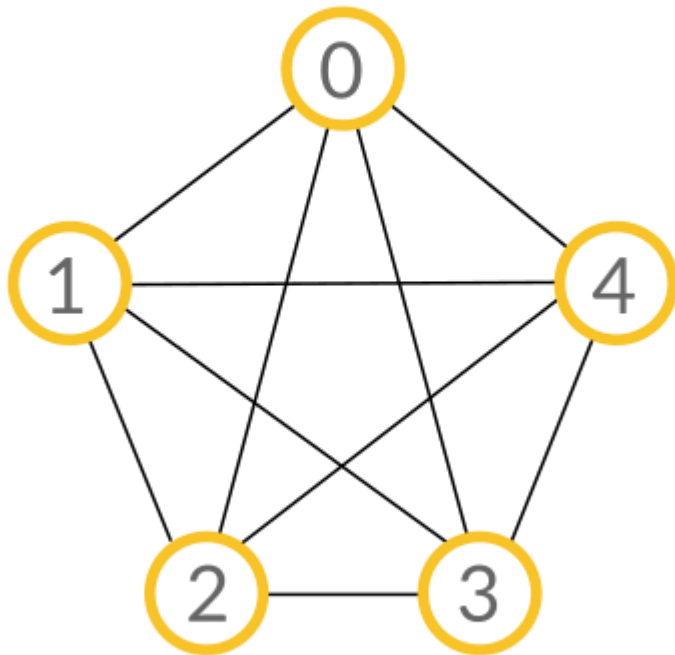
Mac OS: mpicc / mpirun

Communicator(s) & Groups



MPI_COMM_WORLD & SIZE & RANK

MPI_COMM_WORLD

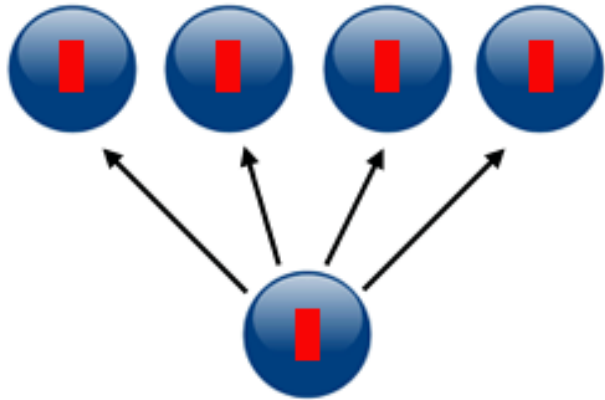


MPI_Comm_rank
MPI_Comm_size

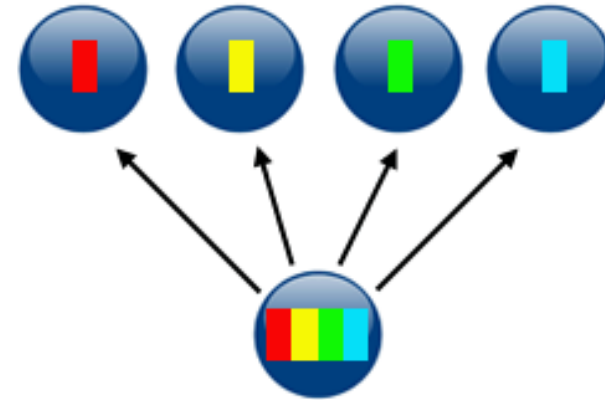
Tipos de Comunicación

- Punto a Punto
- Colectivo

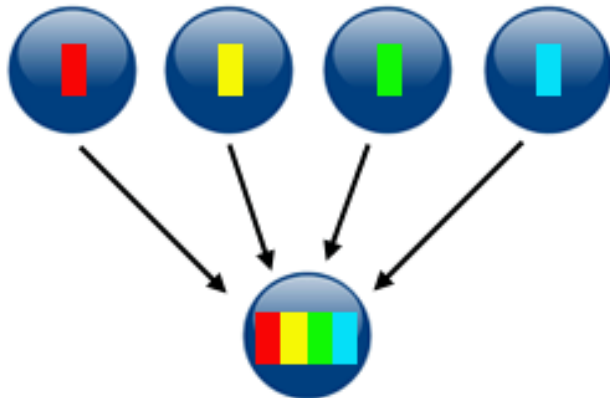
Rutinas de Comunicación Colectiva



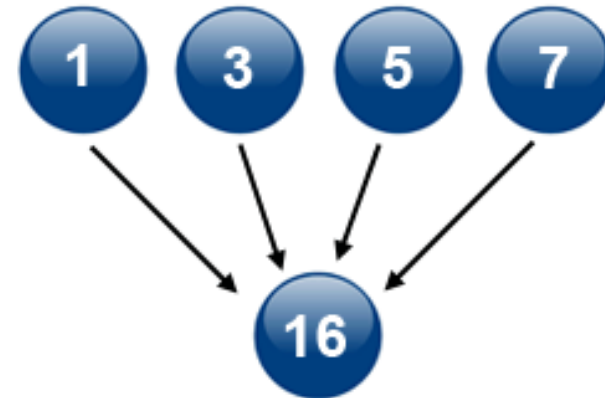
broadcast



scatter



gather



reduction

Estructura de un programa MPI

MPI include file

Declarations, prototypes, etc.

Program Begins

·
·
·

Serial code

Initialize MPI environment

Parallel code begins

·
·
·

Do work & make message passing calls

·
·
·

Terminate MPI environment

Parallel code ends

·
·
·

Serial code

Program Ends

Algunas constantes y comandos **MPI** básicos

MPI_MAX_PROCESSOR_NAME

MPI_COMM_WORLD

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &process_id);

MPI_Comm_size(MPI_COMM_WORLD, &num_processes);

MPI_Get_processor_name(host_name, &name_length)

MPI_Send(&process_id, 1, MPI_INT, partner, 1, MPI_COMM_WORLD);

MPI_Recv(&message, 1, MPI_INT, partner, 2, MPI_COMM_WORLD, &status);

Envío de mensajes bloqueante con **MPI**



MPI_Send



```
int MPI_Send(const void *buf,  
             int count,  
             MPI_Datatype datatype,  
             int dest,  
             int tag,  
             MPI_Comm comm)
```

Recepción de mensajes bloqueante con **MPI**

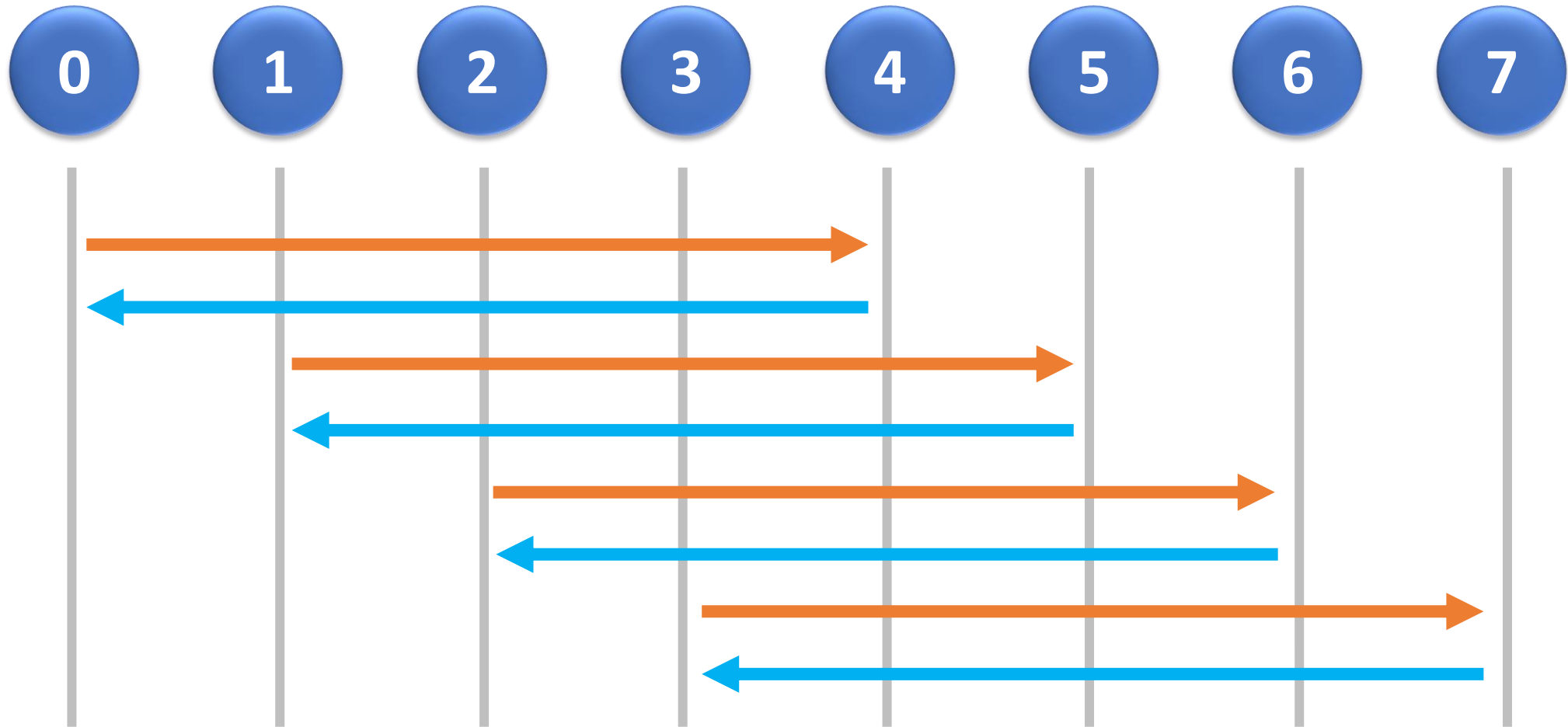


MPI_Recv



```
int MPI_Recv(void *buf,  
             int count,  
             MPI_Datatype datatype,  
             int source,  
             int tag,  
             MPI_Comm comm,  
             MPI_Status *status)
```

Ejercicio **MPI** con interacción bloqueante



Tipos de *datos* MPI

Tipo de Dato MPI	Equivalente en C++
<i>MPI_SHORT</i>	short int
<i>MPI_INT</i>	int
<i>MPI_LONG</i>	long int
<i>MPI_LONG_LONG</i>	long long int
<i>MPI_UNSIGNED_CHAR</i>	unsigned char
<i>MPI_UNSIGNED_SHORT</i>	unsigned short int
<i>MPI_UNSIGNED</i>	unsigned int
<i>MPI_UNSIGNED_LONG</i>	unsigned long int
<i>MPI_UNSIGNED_LONG_LONG</i>	unsigned long long int
<i>MPI_FLOAT</i>	float
<i>MPI_DOUBLE</i>	double
<i>MPI_LONG_DOUBLE</i>	long double
<i>MPI_BYTE</i>	char

Envío de mensajes **NO** bloqueante con **MPI**



MPI_Isend



```
int MPI_Isend(const void* buf,  
             int count,  
             MPI_Datatype datatype,  
             int dest,  
             int tag,  
             MPI_Comm comm,  
             MPI_Request *request)
```

Recepción de mensajes **NO** bloqueante con **MPI**



MPI_Irecv



```
int MPI_Irecv(void* buf,  
              int count,  
              MPI_Datatype datatype,  
              int source,  
              int tag,  
              MPI_Comm comm,  
              MPI_Request *request)
```

Esperar por mensajes **NO** bloqueantes con **MPI**

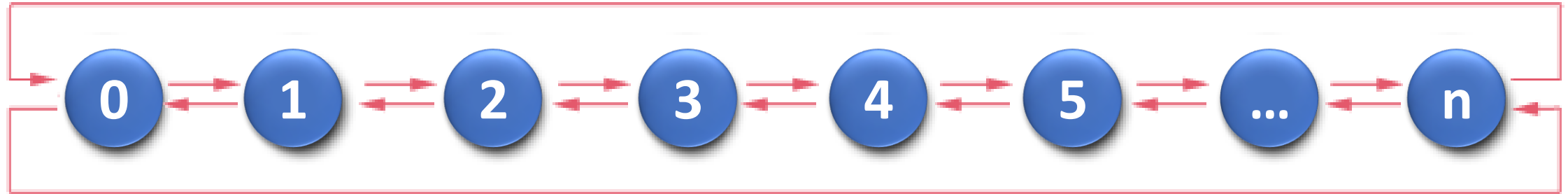


MPI_Wait

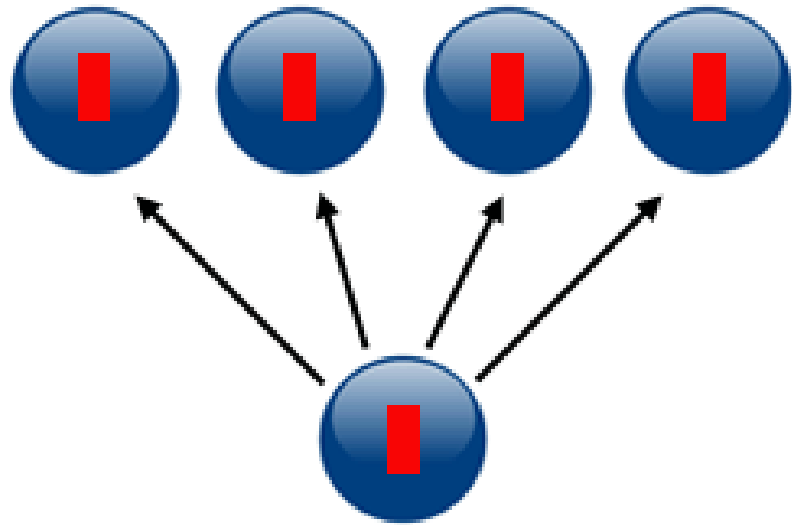


```
int MPI_Wait(MPI_Request *request,  
             MPI_Status *status)
```

Ejercicio **MPI** con interacción **NO** bloqueante



Rutinas de Comunicación Colectiva



broadcast

```
MPI_Bcast(  
    &my_message,  
    1,  
    MPI_INT,  
    process_id,  
    comm_world)
```

Elementos a enviar

Rutinas de Comunicación Colectiva

```
const int NUM_PROCESSES = 3; // NUM_VECTORS
```

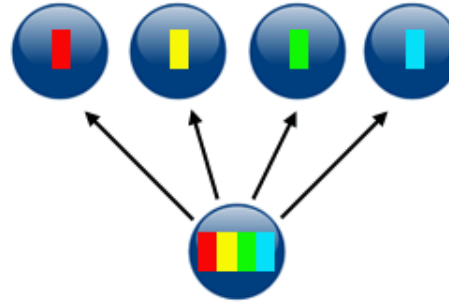
```
const int VECTOR_SIZE = 5;
```

```
float data[NUM_PROCESSES][VECTOR_SIZE] =  
    {{1.0, 2.0, 3.0, 4.0, 5.0},  
     {6.0, 7.0, 8.0, 9.0, 10.00},  
     {11.0, 12.0, 13.0, 14.0, 15.0}};
```

```
float message[VECTOR_SIZE];
```

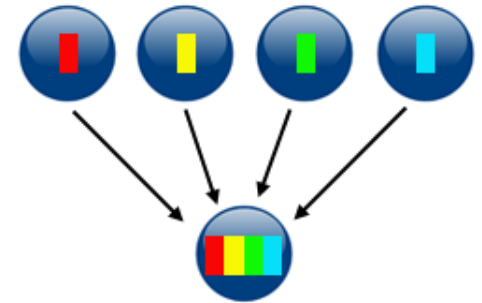
```
float average = 0.0;
```

```
float data_to_be_collected[NUM_PROCESSES];
```



scatter

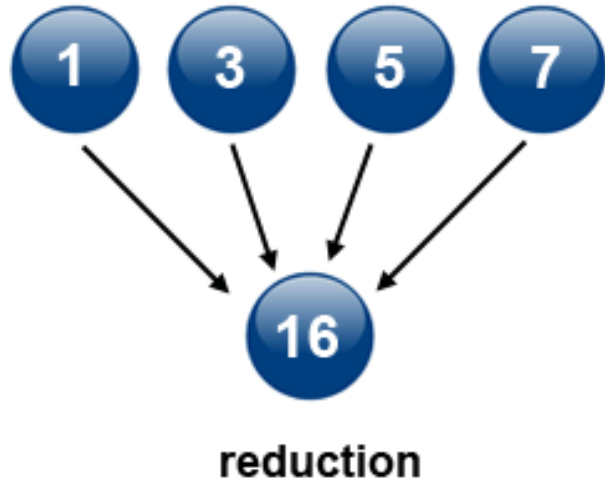
```
MPI_Scatter(  
    data,  
    VECTOR_SIZE,  
    MPI_FLOAT,  
    message,  
    VECTOR_SIZE,  
    MPI_FLOAT,  
    process_id,  
    comm_world)
```



gather

```
MPI_Gather(  
    &average,  
    1, Elementos a enviar  
    MPI_FLOAT,  
    data_to_be_collected,  
    1, Elementos a recibir  
    MPI_FLOAT,  
    process_id,  
    comm_world)
```

Rutinas de Comunicación Colectiva



Operaciones de
Reducción, e.g.,
suma

```
MPI_Reduce(  
    &average,  
    &reduction_result,  
    1,  
    MPI_FLOAT,  
    MPI_OP,  
    process_id,  
    comm_world)
```

Elementos a enviar

Operaciones **MPI REDUCE**

MPI_MAX	maximum	integer, float
MPI_MIN	minimum	integer, float
MPI_SUM	sum	integer, float
MPI_PROD	product	integer, float
MPI LAND	logical AND	integer
MPI_BAND	bit-wise AND	integer, MPI_BYTE
MPI_LOR	logical OR	integer
MPI_BOR	bit-wise OR	integer, MPI_BYTE
MPI_LXOR	logical XOR	integer
MPI_BXOR	bit-wise XOR	integer, MPI_BYTE

Ejercicio **MPI** – Números Primos

Rango	Cantidad de primos
[2, 10]	4
[2, 100]	25
[2, 1000]	168
[2, 10000]	1,229
[2, 100000]	9,592
[2, 1000000]	78,498
[2, 10000000]	664,579
[2, 100000000]	5,761,455
[2, 1000000000]	50,847,534



Ejercicio **MPI** – Números Primos

```
PS C:\Users\Octavio\Desktop\initial_mpi> mpiexec -n 32 primes
10 4 0.0021613
100 25 0.0001208
1000 168 0.0001775
10000 1229 0.0005261
100000 9592 0.0038235
1000000 78498 0.0395258
10000000 664579 0.733421
100000000 5761455 20.2469
1000000000 50847534 618.389
```

Tiempo de
ejecución en
segundos

Rango en dónde se
buscan números
primos

Rango = $[2, 10^n]$

$n = 1..9$

Cantidad de
números primos en
el rango

Tips para el Ejercicio **MPI** – **Números Primos**

Cómo medir el tiempo de ejecución en segundos de una sección de código de un programa de **MPI en C++**

```
#include <mpi.h>
...
    double start;
    double end;

    if (id == 0) start = MPI_Wtime();

    /* INCLUIR CÓDIGO A CRONOMETRAR */

    if (id == 0) end = MPI_Wtime();
...
    cout << "Elapsed time in seconds: " << end - start << endl;
...
```

Tips para el Ejercicio **MPI** – **Números Primos**

Cómo medir tiempo de ejecución en segundos de una sección de código de un programa serial en C++

```
#include <chrono>
```

```
...
```

```
    auto start = chrono::high_resolution_clock::now();
```

```
    /* INCLUIR CÓDIGO A CRONOMETRAR */
```

```
    auto end = chrono::high_resolution_clock::now();
```

```
    auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
```

```
    cout << "Elapsed time in seconds: " << (float)duration.count()/1000000 << endl;
```

```
...
```

Tips para el Ejercicio **MPI** – Números Primos

Qué funciones de comunicación de MPI se pueden utilizar:

MPI_Bcast() Para comunicar a todos los procesos el valor máximo del rango en el que se buscarán los números primos, por ejemplo, 10, 100, 1000, ..., 1000000000.

MPI_Reduce() Con la operación **MPI_SUM** para permitir que cada proceso busque de forma independiente si ciertos números son primos o no y llevar un conteo asociado que eventualmente será acumulado.

Tips para el Ejercicio **MPI** – Números Primos

Cuestiones algorítmicas para el **cálculo de los números primos**

Para determinar si un número es primo

`possible_prime % divisor == 0`

Donde *divisor* va de **2** a $\sqrt{\text{possible_prime}}$

Tips para el Ejercicio **MPI** – Números Primos

Cuestiones algorítmicas para **repartir el trabajo** entre los procesos

Ejemplo para 4 procesos con identificadores 0, 1, 2 y 3

Proceso 0: $2 + 0 = 2$ (possible_prime)

Proceso 0: $6 + 0 = 6$ (possible_prime)

Proceso 0: $10 + 0 = 10$ (possible_prime)

Proceso 1: $2 + 1 = 3$ (possible_prime)

Proceso 1: $6 + 1 = 7$ (possible_prime)

Proceso 1: $10 + 1 = 11$ (possible_prime)

Proceso 2: $2 + 2 = 4$ (possible_prime)

Proceso 2: $6 + 2 = 8$ (possible_prime)

Proceso 2: $10 + 2 = 12$ (possible_prime)

Proceso 3: $2 + 3 = 5$ (possible_prime)

Proceso 3: $6 + 3 = 9$ (possible_prime)

Proceso 3: $10 + 3 = 13$ (possible_prime)

...

Tips para el Ejercicio **MPI** – Números Primos

Posibles funciones matemáticas a utilizar

```
#include <cmath>
```

```
...
```

```
double trunc()
```

```
double sqrt()
```

```
Operador módulo (%)
```

Ejercicio **MPI** – Números Primos con 32 procesos

